WageLens: auditing India's new "wages" definition across a messy payroll estate

Area: payroll taxes and statutory liabilities (PF, gratuity) under India's Labour Codes · Jurisdiction: India, central rules · Prototype: clickable app, deterministic engine, 15 tests

1. The area, and what surprised me

India's four Labour Codes took effect on 21 November 2025. Section 2(y) of the Code on Wages, 2019 defines wages as all remuneration, including basic pay, dearness allowance and retaining allowance, minus exclusions (a) to (k). A proviso adds the twist: if the items in clauses (a) to (i) (statutory bonus, amenities, employer PF, conveyance and LTA, special expenses, HRA, award amounts, overtime, commission) exceed **half of total remuneration**, the excess goes back into wages. Employers compute PF and gratuity on that wage, so a salary built on a 30–40% basic now carries a bigger statutory base.

Three findings surprised me. **The money is large and already booked.** TCS, Infosys and HCLTech took ₹4,373 crore of one-time charges in Q3 FY26, mostly for past-service gratuity and leave (Fig. 2). **Classification decides the answer, and arithmetic barely matters.** A "Special Allowance" or "Flexible Allowance" can land on either side of the line. The Supreme Court's 2019 *Vivekananda Vidyamandir* ruling treats allowances paid to every employee as basic wages for PF, so each call needs judgement and evidence. **Tresata's own job notification makes a clean test case** (Fig. 3). Its CTC sheet shows Basic ₹9,00,000 of ₹18,00,000 and a gratuity line of ₹43,269, which equals ₹75,000 × 15/26. Tresata accrues gratuity on basic alone. If the ₹30,894 monthly Flexible Allowance counts as excluded LTA or conveyance, exclusions reach 48.8% and the structure clears the line by ₹1,803 a month. If it counts as wages, the gratuity base rises 41%.

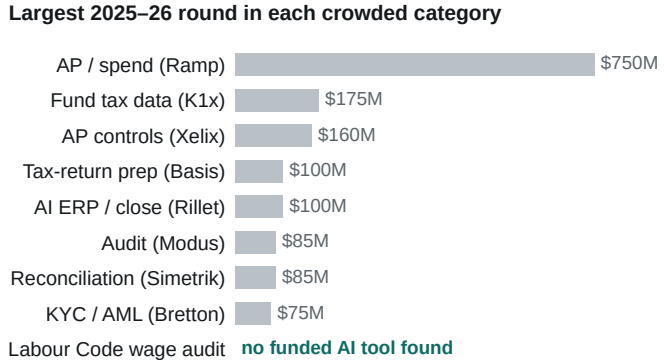


Figure 1. My research map. Every obvious finance-AI idea already has a funded leader; I found no AI tool for the Labour Code wage audit.

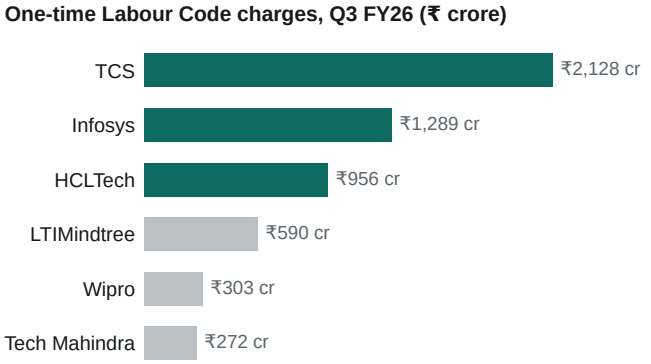


Figure 2. What the new wage definition cost six IT employers in a single quarter (company results, compiled Jan 2026).

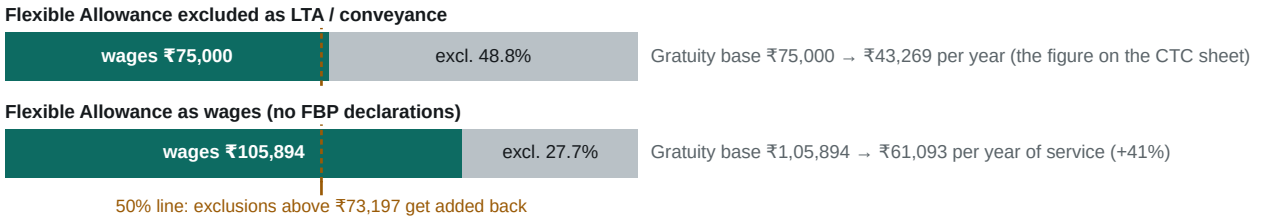


Figure 3. Tresata's published CTC, one month. Same ₹1,46,394, two labels for one allowance, two different gratuity bases.

2. Problem statement

Who	Payroll and compensation heads at mid-to-large Indian employers, and the labour-law CA or statutory auditor who signs off the PF base and the Ind AS 19 gratuity provision. Groups with several payroll systems (acquired entities, outsourced shift payroll, legacy SAP wage types) feel it most.
Task	Classify every pay component against s.2(y), apply the 50% test per employee per wage period, recompute PF and gratuity bases, size the liability change, and show an auditor why each component landed where it did.
How often	Once at transition, then every pay cycle (overtime, commission and bonus months move the ratio), every new component, every increment cycle and every state notification.
Cost today	Spreadsheets per CTC template plus sampled employee checks, rebuilt after each change. Under-paid PF and under-provided gratuity turn into assessments and restatements; over-counting wages inflates cost. Fig. 2 shows the scale for large employers.
Why now	The Codes went live on 21 Nov 2025 and the Centre notified rules on 8 May 2026. MoLE's March 2026 FAQs (as law firms summarise them) apply the new definition to gratuity for every exit after 21 Nov 2025, across all service. By 1 Oct 2026 only 7 to 9 states and UTs had final rules under each Code, so employers will re-run this many times.

Out of scope	Running payroll, state-specific rules (future rule-store versions), leave encashment, actuarial discounting (WageLens hands the actuary the undiscounted statutory base), ESI, bonus eligibility, legal advice.

3. Solution: AI reads the metadata, never the rows

A payroll estate holds millions of employee-month lines but only a few dozen pay components. WageLens points the LLM at those few dozen, where the meaning sits in text, and keeps every rupee calculation in plain code.

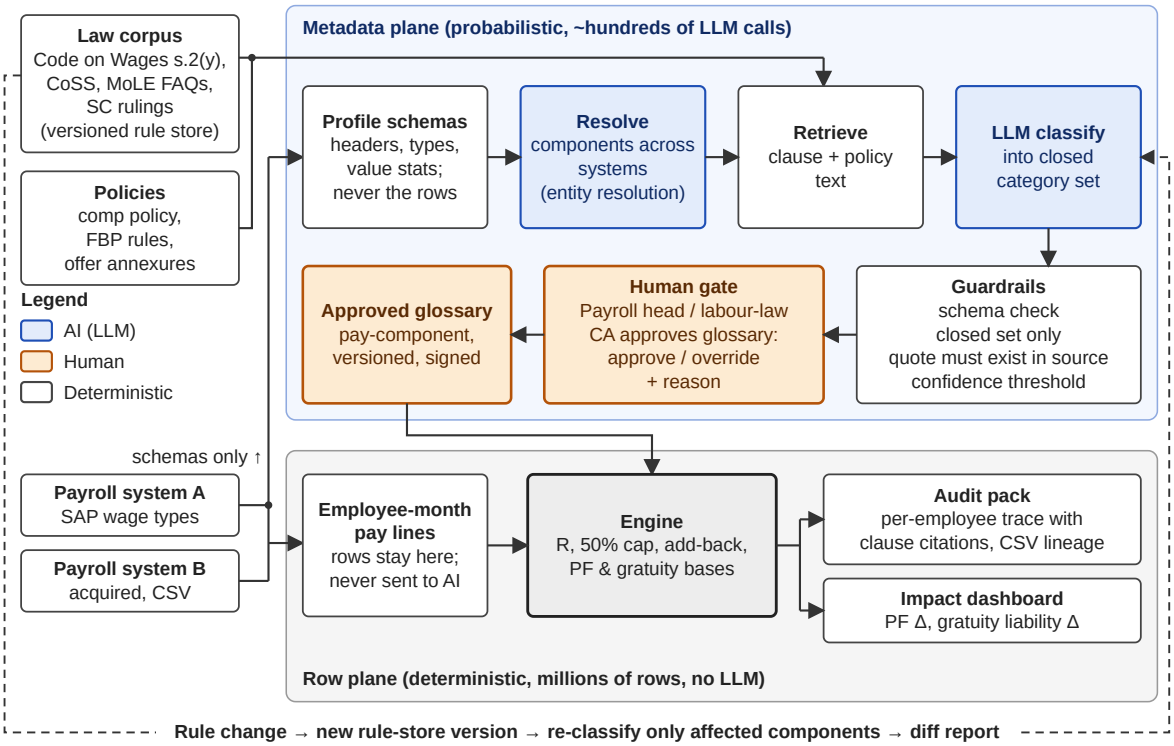


Figure 4. Two planes. The LLM works on component metadata (blue), a person approves the glossary (orange), and code computes every row (grey). Each output traces through the glossary to a clause and a source quote.

Data in. Each payroll system's component dictionary and pay register (codes, labels, headcount paid, amounts); the company's compensation policy, flexible-benefit rules and offer annexures; a versioned law corpus (s.2(y), Code on Social Security, MoLE FAQs, case notes).

Metadata plane. An agent profiles each system, merges duplicates across systems ("1020 SPL ALW" in SAP and "SPECIAL_ALLOW" in the acquired CSV) and keeps look-alikes apart ("OT 2X" and "Night Shift Allow"). It retrieves the clause and policy text, then asks the LLM to place each component in a **closed set of 14 categories** with a rationale and verbatim evidence. Guardrails reject any category outside the set, any quote that does not appear character for character in a retrieved source, and any answer without a citation. The agent retries or escalates.

Human gate. The payroll head or CA reviews the glossary in risk order: lowest confidence first, weighted by the rupee swing if the label flipped (the engine computes that counterfactual). They approve, or override with a written reason. The engine trusts only the approved, versioned glossary.

Row plane. Code applies s.2(y) to every employee-month: total remuneration R, the capped exclusions, the 50% cap, the add-back and the Code wage, then employer PF old and new (statutory ceiling ₹15,000, ₹25,000 from 17 Sep 2026) and the gratuity base (15/26 × wage × completed years, capped at ₹20 lakh). No LLM touches a salary. The browser prototype runs 10,000 employees × 3 months in 79 ms; production runs the same logic as a Spark job in the warehouse.

Outputs. An impact dashboard (add-back headcount, PF Δ per month and year, gratuity liability Δ by entity and department) and an **audit pack**. Each employee-month links line → component → glossary entry (category, clause, quote, approver, time) → ruleset version → formula, so an auditor can recompute any figure by hand.

4. Why AI, and where not

A lookup table keyed on component names handles "HRA". It breaks on terse legacy codes, free-text headers from an acquired system, a "Flexible Allowance" defined in an FBP policy, and allowances whose status turns on policy facts read against case law. Reading and judging that text suits an LLM. The 50% test, PF, gratuity and totals stay in code with hand-computed tests. With the LLM working on 20 to 60 canonical components, AI cost stays fixed per company, and a re-run after a rule change costs nothing.

Where knowledge lives. Law and parameters (threshold, PF ceiling, gratuity cap) sit in an effective-dated rule store. A human owns the company-specific meaning in the approved glossary. The model contributes language understanding and holds no rule. I hit a stale rule while building: EPFO raised the wage ceiling from ₹15,000 to ₹25,000 from 17 September 2026. WageLens handles that as rule-store version v2. December to February periods still compute under v1, and a diff lists the capped-policy employees whose PF moves. A new court ruling would flag the affected categories and send only those components back for review.

Alternatives I rejected. (1) *An LLM per employee or payslip*: millions of calls, numbers that drift between runs, nothing to audit. (2) *Keyword mapping*: silent errors on custom components and no evidence trail. (3) *A RAG chatbot for HR*: it answers questions and never produces the liability or the audit pack. (4) *Fine-tuning a classifier*: no labelled data, and the law moves; each approved glossary becomes labelled data later. (5) *A free-roaming agent doing the maths*: arithmetic stays in code.

5. Uncertainty, risks and the weakest point

Risk	How the design handles it
Wrong classification	Closed category set; quote verification; confidence and ₹-impact ordering; unapproved labels only run as a watermarked DRAFT; overrides need a reason; approved glossaries become the regression eval for the next run.
Wrong merge across systems	Each merge carries its own confidence and evidence (label, co-occurrence, amount profile); low scores go to review. The prototype shows one rejected merge (overtime vs night shift).
Outdated rules	Effective-dated rule store; state rules flagged as not modelled; a change triggers a re-run and an employee-level diff.
Confidential payroll data	WageLens runs inside the customer's VPC, as Tresata's FDEs deploy. The model sees component dictionaries and policy text, never names or salaries, so a small self-hosted model works.
Liability and legal advice	WageLens drafts; the payroll head or CA decides. The screen labels contested items and shows both readings. The system records each approval with name and time.

Weakest point. The law leaves some components open: flexi baskets without per-employee declarations, night-shift allowance, a contractual "retention bonus". The AI cannot settle them. It puts them first in the queue and shows the rupee cost of each reading. One assumption also flips results: whether employer PF counts in "all remuneration" (test T08). WageLens exposes it as a versioned toggle the reviewer signs off.

6. Prototype and results

The clickable app (link in the submission) runs Connect estate → Agent run → Review glossary → Results → Rules & tests. Reviewers can classify a new component live through OpenRouter or Claude, behind the same guardrails. The synthetic estate covers a fictional company with two payroll systems, 33 raw codes, 20 canonical components and 1,200 to 10,000 generated employees over three months.

Tests	**15 of 15 engine tests pass.** I computed every expected value by hand before any code existed. The cases cover Tresata's CTC with the Flexible Allowance as wages and as excluded (T01, T02), the 30%-basic case (T03), special allowance as wages (T04), overtime, commission and bonus months (T05, T06, T09), the PF ceiling (T07), the employer-PF assumption (T08), exit gratuity outside the cap (T10), bill-based reimbursements (T11), cross-system merges (T12), a 40% what-if (T13) and the real Sep 2026 ceiling change (T15). Fourteen integrity checks also pass, including all 41 evidence quotes matching their sources verbatim.
Classifier	18 of 20 components match the human labels (90%). I kept the two misses in the eval.
Scale	33 raw codes resolve to 20 components (13 cross-system merges, 1 rejected). 182,069 pay lines for 10,000 employees compute in 79 ms in a browser. The recorded agent run used 23 steps and about 52k tokens, roughly $0.29 for the whole estate at $3 / $15 per million tokens.
Demo impact	Of 1,200 employees, 248 (20.7%) get an add-back. Employer PF rises ₹18.4 lakh a month and undiscounted gratuity liability rises ₹13.6 crore.
Speed-up	The reviewer makes 20 decisions once, with evidence on screen, in place of re-checking 1,200 salary annexures at every change. A rule change costs a re-run and a diff. My estimate: about 30 minutes of review against about 80 hours of spreadsheet checks.

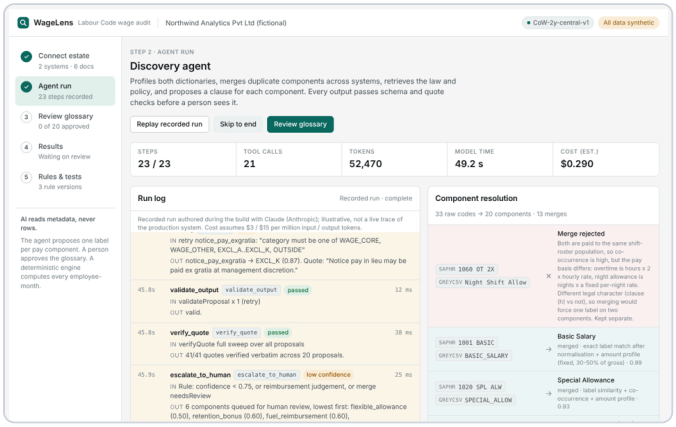
Where it fails. The recorded classifier misses two components. It reads night-shift allowance as a special-expense exclusion, and a contractual monthly retention bonus as a statutory bonus. The review queue puts both on top. The flexi basket needs per-employee FBP declarations; until then the prototype defaults it to wages at low confidence. All data and company documents are synthetic, and the prototype skips state rules.

Before spending more time I would sit with two payroll heads and a labour-law CA to test the category set and the review screen, and check how far Keka, greytHR, Zoho and Darwinbox go for single-system customers (my gap is multi-system estates and independent audit). I would add per-employee FBP declarations as a second, small document stream and try a

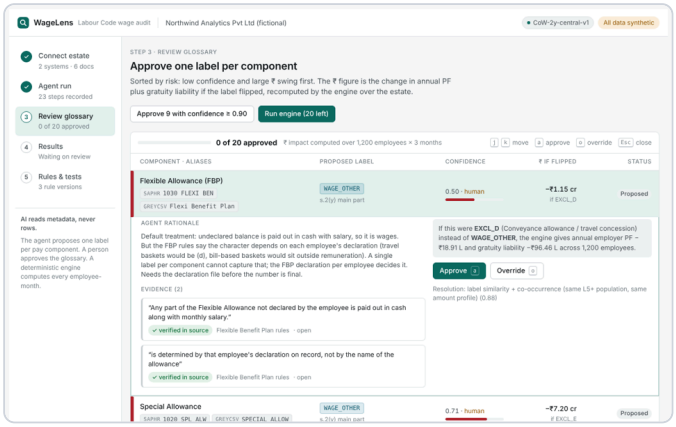
second jurisdiction. The US "no tax on overtime" W-2 code TT for 2026 wages has the same shape: classify pay codes and union-contract clauses once, compute the FLSA premium on every row.

Sources. Code on Wages 2019 s.2(y), bare act (advocatekhoj.com) · Codes in force 21 Nov 2025 (EY India alert) · Central rules 8 May 2026, state status 1 Oct 2026 (AMSS Employment Law Bulletin, Oct 2026) · Q3 FY26 charges (company results compiled by CA Rajput, 17 Jan 2026; People Matters, 16 Jan 2026) · RPFC (II) West Bengal v. Vivekananda Vidyamandir, SC, 28 Feb 2019 · EPFO ceiling ₹25,000 from 17 Sep 2026 (Business Today, 16 Sep 2026; TeamLease RegTech) · MoLE Labour Code FAQs, Mar 2026 · Funding rounds in Fig. 1: TechCrunch, FinTech Global, company announcements, 2025–26 · Tresata CTC: Tresata's 2026 campus job notification.

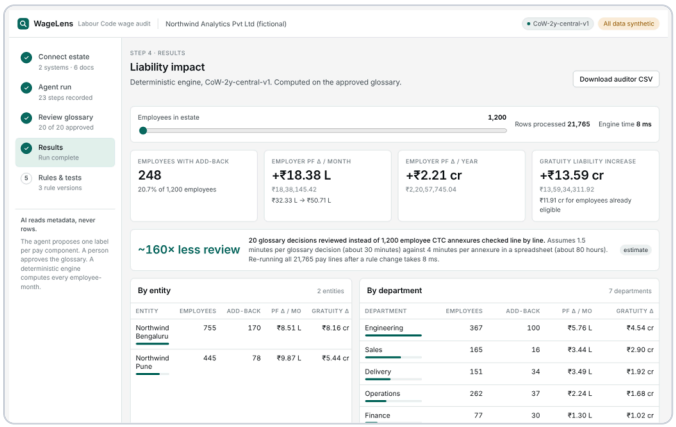
7. The prototype in four screens



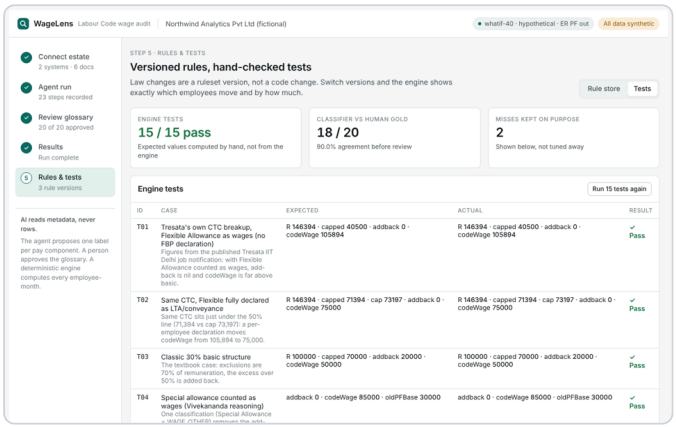
Agent run. The agent plans, profiles, resolves, retrieves and classifies. Guardrail events show a rejected merge, a quote not found and an invented clause "(l)", with token and cost counters. The agent sees metadata only.



Review glossary. Components sorted by risk and by the rupee swing if the label flipped. Each quote carries a "verified in source" badge. Approve, or override with a reason (keys j, k, a, o).



Results. Code runs every employee-month and reports in lakh and crore. Click any employee for a line-by-line trace with clause citations, or download the auditor CSV.



Rules & tests. 15/15 hand-computed engine tests, 18/20 classifier agreement with the two misses kept on purpose; the Rule store tab swaps versions (v1 → real Sep 2026 EPF ceiling → hypothetical 40%) and shows an employee-level diff.

Appendix: what I added beyond the AI

Where AI helped

- **Discovery.** I ran five research agents in parallel (recent YC batches, 2025–26 funded finance-AI startups, practitioner voices, Indian rule changes, new 2025–26 regimes elsewhere) and had them map the crowded ideas. They returned about 60 candidate problems and 20 categories to avoid (Fig. 1).
- **Fit to the company.** I gave the AI Tresata's two job descriptions (FDE, and Software Engineer, Data for Agents) and asked which problem would show agents working inside a customer's messy data estate. A document chatbot would show none of that.
- **Build.** Coding agents wrote the engine, the synthetic payroll estate and the UI in parallel against an interface contract I approved. I fixed the hand-computed test values before any code existed.

Key prompts. (1) "Research problems in finance that are still open… from recent YC or recently funded AI finance companies, podcasts or Twitter. Don't pick any famous idea that has been solved in many places." (2) "Here are the company and job descriptions; think of problems they would be interested in and impressed by; merge or combine ideas." (3) The production classification prompt in `prompts/classify.md`: closed category list, verbatim evidence, JSON output, lower confidence when unsure.

What I rejected or corrected

- **Crowded ideas the AI ranked high.** One research pass put Form 6765 Section G (US R&D credits) first. The funded-startup pass showed Onshore, MainStreet and Boast already own R&D credits. IEEPA tariff refunds had the strongest "why now", but they sit next to the brief's excluded rules-of-origin example, so I dropped them.
- **The model doing the maths.** Early designs let the LLM compute the add-back. I moved every number into deterministic code and kept the LLM on component meaning.
- **"Special allowance is an exclusion."** Many salary-restructuring explainers treat every non-basic allowance as excluded. Section 2(y) counts all remuneration unless a clause excludes it, and the Supreme Court's 2019 PF ruling points the same way for allowances paid to everyone. I made "other wages" the default and show the evidence.
- **A stale rule in my own spec.** The first engine spec hard-coded the ₹15,000 EPF ceiling from memory. A fact-check found EPFO raised it to ₹25,000 from 17 Sep 2026. I turned the constant into an effective-dated rule-store version, and it became the prototype's real "rules changed" demo.
- **A hidden assumption.** Whether employer PF sits inside "all remuneration" changes results (test T08). I made it a visible ruleset setting the reviewer signs off, so the code never picks it silently.

What came from me

- **The problem.** I picked the Labour Code wage audit over higher-scoring US and UK options. I can check it against Indian salary structures I know, and it suits a data-platform company better than a document-reading problem.
- **A product over a notebook.** I chose a clickable product built around the review screen. The hard part is a person deciding 20 labels fast and with confidence; the arithmetic takes one function.
- **The process.** I planned first, then ran parallel agents against a fixed interface and hand-computed expectations, so the AI never graded its own arithmetic.
- **The job notification as test T01.** Tresata's gratuity line of ₹43,269 equals ₹75,000 × 15/26, so Tresata accrues gratuity on basic alone. I checked it by hand. The 41% swing from relabelling one allowance shows the problem in a single example (Fig. 3).

One thing I'd defend live without AI

The s.2(y) computation, and why the AI sits on the metadata. For one employee-month, total remuneration R equals wages (basic, DA, retaining allowance and any other unexcluded pay) plus the clause (a) to (i) exclusions. If those exclusions exceed 50% of R, the excess joins wages. Take basic ₹30,000, HRA ₹30,000, conveyance ₹20,000 and LTA ₹20,000: R is ₹1,00,000, exclusions ₹70,000, the cap ₹50,000, the add-back ₹20,000 and the Code wage ₹50,000. Gratuity then uses ₹50,000 × 15/26 per completed year in place of ₹30,000 × 15/26. That formula runs the same way on every row. The meaning of "SPL ALW" or "Flexi Benefit Plan" changes from company to company, so I put the LLM there, and a person approves its answer once before the engine touches a row.